

Práctica de Investigación:

Arquitectura y Tecnologías de una Plataforma Web de Gran Escala

Plataforma analizada: **Instagram**

Módulo: Despliegue de Aplicaciones Web
Formato de entrega: Documento en PDF

Autor: **Iker Ríos Navarro**
Curso 2025/2026

ÍNDICE

1. Introducción
2. Modelo arquitectónico general
3. Tecnologías Front-end
4. Tecnologías Back-end
5. Sistemas de bases de datos y almacenamiento
6. Comunicación entre servicios y APIs
7. Infraestructura, escalabilidad y despliegue
8. Diagrama arquitectónico (descripción)
9. Conclusión
10. Referencias (formato APA 7)

1. Introducción

En esta práctica se analiza la arquitectura y el stack tecnológico de **Instagram** como ejemplo de plataforma web de gran escala. Instagram es una red social centrada en la publicación de fotografías, vídeos, historias y contenido vertical. La plataforma da servicio a cientos de millones de personas de forma continua, lo que requiere una arquitectura distribuida, resiliente y preparada para escalar rápidamente.

El objetivo del trabajo es revisar cómo se estructura la plataforma a nivel de Front-end, Back-end, bases de datos, comunicación entre servicios e infraestructura, aplicando los conceptos estudiados en clase sobre arquitecturas modernas basadas en microservicios.

2. Modelo arquitectónico general

Instagram utiliza una **arquitectura de microservicios** desplegada mediante contenedores. Cada módulo del sistema opera como un servicio independiente: autenticación, feed, historias, mensajería, notificaciones, estadísticas, compresión de imágenes y más.

Este modelo es adecuado porque:

- **Escala horizontalmente:** permite aumentar instancias solo en los servicios con mayor demanda.
- **Aumenta la resiliencia:** un fallo en un servicio no detiene toda la plataforma.
- **Mejora el mantenimiento:** los equipos trabajan en servicios aislados.
- **Reduce la latencia:** se despliegan servicios en distintos centros de datos.

Además, para eventos internos se utilizan patrones orientados a eventos, que facilitan el procesamiento asíncrono.

3. Tecnologías Front-end

Instagram utiliza varios frentes tecnológicos para sus interfaces:

- **Web:** React + JavaScript/TypeScript, con componentes reutilizables y renderizado eficiente.
- **Móvil:** apps nativas desarrolladas en Swift (iOS) y Kotlin (Android), optimizadas para cámara, edición y animaciones.

Estas tecnologías permiten:

- Interactividad fluida gracias a React.
- Rendimiento alto en scroll, historias y edición.
- Precarga de contenido para reducir tiempos de espera.
- Diseño coherente gracias al uso de componentes modulares.

4. Tecnologías Back-end

Instagram combina **Python con Django** (servicios clásicos) y **Go** (servicios de alta concurrencia).

Servicios principales:

- Autenticación y control de sesiones.
- Generación del feed mediante algoritmos ML.
- Sistema de mensajería en tiempo real.
- Procesamiento de imágenes y vídeos.
- Servicios de moderación y detección de contenido no permitido.

Para análisis masivo se emplean motores como Spark y sistemas internos de Meta.

5. Sistemas de bases de datos y almacenamiento

Instagram usa múltiples sistemas según el tipo de dato:

- **SQL (PostgreSQL/MySQL):** datos críticos como usuarios y configuraciones.
- **NoSQL (Cassandra):** likes, seguidores, visualizaciones y eventos rápidos.
- **Redis:** caché para acelerar carga del feed y datos repetidos.
- **Almacenamiento distribuido tipo S3:** imágenes y vídeos replicados y servidos vía CDN.

Cada sistema está optimizado para su tipo de carga y volumen.

6. Comunicación entre servicios y APIs

- **REST interno:** usado para la mayoría de microservicios.
- **gRPC:** empleado para llamadas internas de baja latencia.
- **Kafka:** sistema de mensajería para eventos como nuevas publicaciones o notificaciones.

Las APIs externas son limitadas por motivos de seguridad, pero las internas coordinan todo el funcionamiento de la plataforma.

7. Infraestructura, escalabilidad y despliegue

Instagram opera sobre centros de datos de Meta. Sus pilares técnicos:

- **Docker:** empaquetado de microservicios.
- **Kubernetes:** orquestación, escalado automático y recuperación de fallos.
- **CDN global:** distribución de contenido multimedia para reducir latencia.
- **Despliegue continuo:** actualizaciones graduales para detectar errores antes de liberar globalmente.

Estas técnicas permiten absorber picos de tráfico sin afectar la calidad del servicio.

8. Diagrama arquitectónico (descripción)

Flujo simplificado de una petición:

1. El usuario abre la app y envía una petición.
2. El balanceador dirige la petición al API Gateway.
3. El Gateway la envía al microservicio correspondiente (p. ej., Feed).
4. El servicio consulta Redis (caché) y bases SQL/NoSQL.
5. Para multimedia, se generan URLs hacia el almacenamiento distribuido y CDN.
6. Se construye la respuesta y se devuelve al cliente.

Este flujo incluye clientes, CDN, balanceadores, gateways, microservicios, bases SQL, NoSQL, cachés y almacenamiento.

9. Conclusión

Instagram demuestra cómo una plataforma global necesita una arquitectura distribuida basada en microservicios, sistemas de almacenamiento especializados y despliegues continuos. Su diseño permite mantener disponibilidad, escalabilidad y rendimiento incluso durante picos de uso extremos. El análisis conecta los conceptos teóricos del módulo con un caso real de producción a escala masiva.

10. Referencias (APA 7)

Meta Engineering. (2023). *Instagram architecture overview*. Meta Platforms.
Meta Engineering. (2023). *Scaling Instagram systems*. Meta Platforms.

Nielsen Norman Group. (2020). *Usability guidelines for social media interfaces*. NNGroup.